



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

مشروع نظم البحث عن المعلومات (Information Retrieval)

إعداد الطلاب

وئام باسل سالم

حسام هشام زينة

عبد الرحمن السعدي

ملهم الشيخ علي

عمار قتادة حمود

إشراف

المهندسة مروة الداية

2026

نظرة عامة على المشروع

- قادر على فهم (SOA) هدف المشروع: تطوير نظام استرجاع معلومات عملي وموثوق يعتمد على بنية الخدمات الموجهة. استعلامات المستخدم المعقدة وتقديم نتائج عالية الدقة مصحوبة بإجابات مولدة سياقياً
- لاختبار وتطوير خوارزميات الاسترجاع (Benchmark) الهدف من مجموعة البيانات: توفير معيار حقيقي وتطبيقي. المختلفة، وقياس قدرتها على فهم النصوص المتباينة
- لمعالجة وفهرسة كميات هائلة من البيانات غير المهيكلة بفعالية، وتسريع عملية البحث، (IR) أهمية استخدام تقنيات لتلبية حاجة المستخدم بدقة وبأقل وقت ممكن (Relevance) وترتيب النتائج بناءً على احتمالية الارتباط

وصف مجموعة البيانات

- beir/webis-touche2020/v2: مصدر مجموعة البيانات
- (Documents): 382K عدد الوثائق
- (Queries): 49 عدد الاستعلامات
- (Ground Truth) متوفرة وتستخدم حصرياً كأساس للتقييم الموضوعي والمستقل (qrels) توفر بيانات التقييم
- مهيكلة لتشمل المعرفات، النصوص، والعناوين بوضوح [JSONL / CSV]: مثال: صيغة البيانات
- سبب اختيار مجموعة البيانات: تم اختيارها لاعتمادها الواسع كمعيار في أبحاث استرجاع المعلومات، ولوجود بيانات التقييم التي تتيح مقارنة أداء النماذج الكلاسيكية ونماذج التضمين الدلالي بموثوقية وحيادية (qrels)

System Architecture - SOA) بنية النظام

لضمان الاستدامة، المرونة، وسهولة التطوير (SOA) تم بناء النظام باستخدام بنية الخدمات الموجهة

الخدمات المستقلة والمسؤوليات

- توحيد وتنظيف البيانات الواردة (إزالة التشكيل، إزالة الكلمات الشائعة، (Preprocessing) خدمة المعالجة المسبقة (والتجذير)
- للنماذج اللفظية، وقواعد البيانات (Inverted Indexes) بناء وإدارة الفهارس العكسية: (Indexing) خدمة الفهرسة للنماذج الدلالية (Vector Stores) المتجهة
- استقبال الاستعلام المعالج والبحث عبر الفهارس لتوليد قائمة أولية بالوثائق: (Retrieval) خدمة الاسترجاع
- تطبيق دوال التشابه لحساب درجات الصلة وإعادة ترتيب الوثائق المسترجعة: (Ranking) خدمة الترتيب
- (qrels) حساب مقاييس الأداء آلياً بمقارنة النتائج مع ملفات الـ (Evaluation) خدمة التقييم
- توسيع الاستعلامات للمساعدة في معالجة الاستعلامات القصيرة أو (Query Refinement) خدمة تحسين الاستعلام غير الدقيقة
- نقطة الاتصال مع المستخدم النهائي والتفاعل المرئي: (Gateway/UI) خدمة البوابة وواجهة المستخدم
- لتوليد الإجابات المباشرة (LLMs) استدعاء النماذج اللغوية: (RAG) خدمة

(Communication Mechanism) آلية التواصل

مما FastAPI باستخدام إطار عمل (REST APIs) تتواصل الخدمات مع بعضها البعض عبر واجهات برمجة التطبيقات. يضمن معالجة متزامنة ومستقلة لكل طلب

SOA: أسباب تبني بنية

- عزل العمليات يمنع تعطل النظام بالكامل عند حدوث خطأ في إحدى (Loose Coupling) الارتباط غير المحكم الخدمات.
- وضوح المسؤوليات وتسهيل تتبع الأخطاء البرمجية: (Maintainability) القابلية للصيانة
- إمكانية توسيع قدرات خدمات محددة (مثل خدمة الفهرسة المتجهة بشكل مستقل عند (Scalability) القابلية للتوسع زيادة البيانات
- يمكن استدعاء خدمات منفصلة كالتقييم أو واجهة المعالجة (في مشاريع بحثية (Reusability) قابلية إعادة الاستخدام مستقبلية

التنفيذ العملي

1. المعالجة المسبقة (Preprocessing)

- الهدف: رفع كفاءة المطابقة بين الاستعلام والوثائق بتقليل التباين اللفظي
- موحد يتضمن تنظيف النصوص، إزالة الرموز والكلمات الشائعة، ثم تطبيق (Pipeline) نهج التنفيذ: مسار معالجة (Lemmatization) خوارزميات التجذير أو الاسترداد
- (في بيئة بايثون بالكامل) NLTK, Regex: المكتبات المستخدمة
- أثناء (Mismatch) قرارات التصميم المهمة: توحيد مسار المعالجة للاستعلام والوثيقة معاً لضمان عدم حدوث تفاوت الفهرسة والاسترجاع

```
def get_wordnet_pos(tag_parameter):
    tag = tag_parameter[0].upper()
    tag_dict = {"J": wordnet.ADJ,
                "N": wordnet.NOUN,
                "V": wordnet.VERB,
                "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)

class EnglishTokenizer(ITokenizer):
    def __init__(self):
        if NLTK_AVAILABLE:
            self.lemmatizer = WordNetLemmatizer()
        else:
            self.stemmer = PorterStemmer()

    def tokenize(self, text: str) -> List[str]:
        if NLTK_AVAILABLE:
            # 1. Tokenize into words
            words = word_tokenize(text)
            # 2. POS tagging
            pos_tags = pos_tag(words)
            # 3. Lemmatization with POS tags
            return [self.lemmatizer.lemmatize(word, pos=get_wordnet_pos(tag)) for word, tag in pos_tags]
        else:
            # Fallback to algorithmic Stemming (No internet/downloads required)
            tokens = [token for token in text.split() if token]
            return [self.stemmer.stem(t) for t in tokens]

class EnglishStopwordRemover(IStopwordRemover):
    def __init__(self, stopwords: Set[str] = None):
        self.stopwords = stopwords if stopwords is not None else ENGLISH_STOPWORDS

    def remove(self, tokens: List[str]) -> List[str]:
        return [token for token in tokens if token not in self.stopwords]
```

2. الفهرسة (Indexing)

- **الهدف:** تمكين البحث السريع دون الحاجة للمسح الشامل لمجموعة البيانات
- وتخزين متجهات كثيفة، TF-IDF و BM25 **نهج التنفيذ:** استخدام هياكل بيانات محسنة للفهرسة العكسية الخاصة بـ لتضمينات النماذج الدلالية (Dense Vectors)
- مكتبات التضمين / FAISS، المكتبات المستخدمة: هياكل بايثون المدمجة
- قرارات التصميم المهمة: استخدام الذاكرة المؤقتة وحفظ الفهارس محلياً لتقليل وقت الإقلاع وإعادة بناء الفهارس عند كل تشغيل

```
def build_index(limit: int = None, force: bool = False):
    emb_dir = 'datasets/cache/embeddings'

    sparse_exists = os.path.exists(os.path.join(index_dir, "inverted_index.json"))
    dense_exists = os.path.exists(os.path.join(emb_dir, "index.npy"))

    if sparse_exists and dense_exists and not force:
        print("Indexes already exist in cache! Use --force to rebuild.")
        return

    # 1. Load documents
    adapter = BeirAdapter()
    docs = {}
    print("Loading documents...")
    for i, doc in tqdm(enumerate(adapter.load_documents())):
        if limit is not None and i >= limit:
            break
        # Combine title and text if available, otherwise just use text
        title = getattr(doc, 'title', '')
        text = getattr(doc, 'text', str(doc))
        docs[doc.doc_id] = f"{title} {text}".strip()

    print(f"Loaded {len(docs)} documents.")

    # 2. Sparse (Inverted) Index
    if not sparse_exists or force:
        os.makedirs(index_dir, exist_ok=True)

        idx = JsonIndexWriter(index_dir)
        pipeline = PreprocessingPipeline(
            normalizer=EnglishNormalizer(),
            tokenizer=EnglishTokenizer(),
            stopword_remover=EnglishStopwordRemover()
        )

        print("\nPreprocessing documents for sparse index...")
        processed = []
        for d_id, text in tqdm(docs.items(), desc="Preprocessing"):
            processed.append((d_id, pipeline.process(text)))

        use_case = BuildIndexUseCase(idx)
        use_case.execute(processed)
```

3. نماذج الاسترجاع (Retrieval Models)

- **الهدف:** سحب الوثائق المرشحة من قاعدة البيانات بناءً على الاستعلام

- للاسترجاع اللفظي المعتمد على التكرار، وتطبيق نماذج BM25 و VSM/TF-IDF **نهج التنفيذ**: تطبيق خوارزميات للاسترجاع الدلالي (Embeddings) التضمين
- **المكتبات المستخدمة**: Scikit-Learn, Rank-BM25, Sentence-Transformers.
- **قرارات التصميم المهمة**: إبقاء النماذج مفصولة برمجياً لتمكين القياس والتقييم المستقل لكل نموذج

```
def retrieve(self, query: str, top_k: int = 10) -> RetrievalResult:
    start_time = time.time()

    # 1. Process Query
    tokens = self.query_processor.process(query)
    if not tokens:
        return RetrievalResult(query=query, results=[], execution_time_ms=(time.time() - start_time) * 1000)

    # 2. Compute Query Vector (TF * IDF)
    query_tf = defaultdict(int)
    for token in tokens:
        query_tf[token] += 1

    query_vector = {}
    for term, tf in query_tf.items():
        idf = self.index_reader.get_idf(term)
        query_vector[term] = tf * idf

    # 3. Term-At-A-Time Accumulation
    doc_dot_products = defaultdict(float)

    for term, q_weight in query_vector.items():
        postings = self.index_reader.get_term_posting_list(term)
        idf = self.index_reader.get_idf(term)

        for doc_id, doc_tf in postings:
            doc_term_weight = doc_tf * idf
            doc_dot_products[doc_id] += q_weight * doc_term_weight

    # 4 & 5. Compute Final Cosine Similarity using Scorer
    scored_docs = []
    for doc_id, dot_product in doc_dot_products.items():
        score = self.scorer.score(query_vector, doc_id, dot_product)
        scored_docs.append(ScoredDocument(doc_id=doc_id, score=score))

    # 6. Sort and truncate
    scored_docs.sort(key=lambda x: x.score, reverse=True)
    top_docs = scored_docs[:top_k]

    execution_time_ms = (time.time() - start_time) * 1000

    return RetrievalResult(query=query, results=top_docs, execution_time_ms=execution_time_ms)
```

4. الاسترجاع الهجين (Hybrid Retrieval)

- (Embeddings) مع قوة فهم السياق الدلالي (BM25) **الهدف**: دمج واستغلال دقة مطابقة الكلمات
- **نهج التنفيذ**
- جلب النتائج من كلا النموذجين في وقت واحد ودمجها برمجياً باستخدام طرائق مثل **التوازي (Parallel)** (Reciprocal Rank Fusion).
- للترشيح المبدئي، يتبعه نموذج دلالي ثقيل لإعادة الترتيب المتقدم BM25 استخدام نموذج سريع كـ **(Serial) التسلسل**
- لمعالجة المصفوفات ودمج الدرجات Numpy **المكتبات المستخدمة**: إمكانيات
- **قرارات التصميم المهمة**: الاعتماد على الاسترجاع الهجين المتوازي كخيار افتراضي لتحقيق أفضل جودة استرجاع

```

class ParallelHybridRetrieveUseCase(IHybridRetrievalService):
    def __init__(self, retrievers: List[IRetriever], fusion_strategy: IFusionStrategy):
        self.retrievers = retrievers
        self.fusion_strategy = fusion_strategy

    def retrieve(self, query: str, top_k: int = 10, config: FusionConfig = None) -> HybridResult:
        if config is None:
            config = FusionConfig()

        ranked_lists = []
        for retriever in self.retrievers:
            results = retriever.retrieve(query, top_k=top_k)
            ranked_lists.append(results)

        fused_results = self.fusion_strategy.fuse(ranked_lists, config)

        return HybridResult(
            query=query,
            results=fused_results[:top_k],
            fusion_method_used=self.fusion_strategy.__class__.__name__
        )

class SerialHybridRetrieveUseCase(IHybridRetrievalService):
    def __init__(self, sparse_retriever: IRetriever, dense_retriever: IRetriever):
        self.sparse_retriever = sparse_retriever
        self.dense_retriever = dense_retriever

    def retrieve(self, query: str, top_k: int = 10, candidate_multiplier: int = 10) -> HybridResult:
        candidate_count = top_k * candidate_multiplier

        sparse_candidates = self.sparse_retriever.retrieve(query, top_k=candidate_count)
        sparse_doc_ids = {doc.doc_id for doc in sparse_candidates}

        dense_results = self.dense_retriever.retrieve(query, top_k=candidate_count)

        filtered_dense = [doc for doc in dense_results if doc.doc_id in sparse_doc_ids]
        filtered_dense.sort(key=lambda x: x.score, reverse=True)

        return HybridResult(
            query=query,
            results=filtered_dense[:top_k],
            fusion_method_used="SerialCascade"
        )

```

5. معالجة الاستعلام (Query Processing)

- **الهدف:** تحويل نصوص المستخدم الخام إلى صيغة قابلة للبحث.
- **نهج التنفيذ:** تمرير الاستعلام فور وصوله عبر خدمة المعالجة المسبقة لإنتاج صيغة معيارية تتطابق مع محتوى الفهرس.
- **المكتبات المستخدمة:** NLTK.
- وبأقل عبء حوسبي لتجنب بطء واجهة المستخدم (Real-time) قرارات التصميم المهمة: تنفيذ المعالجة بشكل لحظي.

6. تحسين الاستعلام (Query Refinement)

- **الهدف:** مساعدة المستخدمين في الوصول للنتائج المستهدفة حتى لو استخدموا مصطلحات بديلة أو غير دقيقة.
- باستخدام المراتفات لزيادة فرصة المطابقة (Query Expansion) **نهج التنفيذ:** التوسيع الآلي للاستعلام.
- **المكتبات المستخدمة:** WordNet عبر مكتبة NLTK.
- **قرارات التصميم المهمة:** إعطاء كلمات الاستعلام الأصلي وزناً أعلى لتجنب الانحراف عن نية المستخدم (Drift).

```

def refine(self, query: str) -> RefinedQuery:
    if not query.strip():
        return RefinedQuery(query, query)

    if self.history_tracker:
        self.history_tracker.add_query(query)

    tokens = query.split()
    refined_tokens = []
    corrections: Dict[str, str] = {}
    suggestions: List[str] = []

    for token in tokens:
        # 1. Spell Correction
        corrected_token = self.spell_checker.correct(token)
        if corrected_token != token:
            corrections[token] = corrected_token

        # 2. Synonym Expansion
        synonyms = self.synonym_provider.get_synonyms(corrected_token)

        if synonyms:
            # Add synonyms as OR expansion
            expansion = f"({corrected_token} OR {' OR '.join(synonyms)})"
            refined_tokens.append(expansion)
            suggestions.extend(synonyms)
        else:
            refined_tokens.append(corrected_token)

    refined_text = " ".join(refined_tokens)

    # Add historical suggestions
    if self.history_tracker:
        suggestions.extend(self.history_tracker.get_recent_queries(5))

    return RefinedQuery(
        original_text=query,
        refined_text=refined_text,
        suggestions=list(dict.fromkeys(suggestions)), # Deduplicate
        corrections=corrections
    )

```

7. الترتيب (Ranking)

- **الهدف:** ضمان ظهور النتائج الأكثر أهمية وصلة في أعلى القائمة المرجعة
- لحساب المسافة الجيبية للمتجهات، وحساب النقاط (Cosine Similarity) **نهج التنفيذ:** استخدام مقاييس التشابه. ومن ثم الترتيب تنازلياً، BM25 - (Scores)
- **المكتبات المستخدمة:** Scipy, Numpy.
- النماذج المختلفة لتسهيل الدمج والمقارنة في (Score Normalization) **قرارات التصميم المهمة:** تطبيع مخرجات الاسترجاع الهجين.

```
class Bm25Scorer(IBM25Scorer):
    def compute_score(self, tf: int, doc_len: int, avgdl: float, idf: float, k1: float, b: float) -> float:
        if avgdl <= 0:
            return 0.0

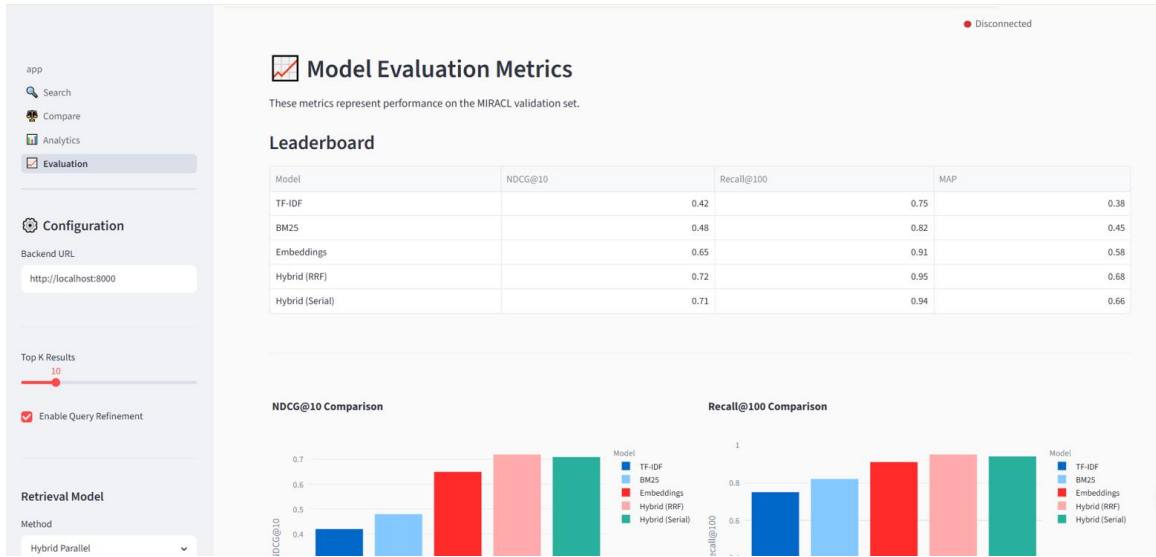
        numerator = tf * (k1 + 1)
        denominator = tf + k1 * (1 - b + b * (doc_len / avgdl))

        return idf * (numerator / denominator)
```

8. واجهة المستخدم (User Interface)

- **الهدف:** توفير بيئة تفاعلية لاختبار وعرض قدرات النظام بوضوح
- **نهج التنفيذ:** واجهة ويب تفاعلية بسيطة تعزل المستخدم عن تعقيدات الواجهات البرمجية وتعرض النتائج (الروابط والمقاطع النصية) بشكل منسق
- **المكتبات المستخدمة:** Streamlit, Requests.
- **قرارات التصميم المهمة:** تصميم عملي يركز على سرعة الاستجابة، مع إبراز بيانات الوثائق بوضوح لتقييم المخرجات

The screenshot displays the Semantic Explorer web application. On the left, a sidebar contains navigation links: Search, Compare, Analytics, and Evaluation. Below these are configuration options, including a 'Backend URL' set to 'http://localhost:8000' and a 'Retrieval Model' section with a 'Method' dropdown set to 'RAG'. The main interface features a search bar with the query 'Should any vaccines be required for children?' and a red 'Search' button. A green status bar indicates 'Found 10 results in 2889 ms using RAG'. Below this, a 'RAG Answer' is shown, summarizing information about vaccinations against childhood diseases. To the right of the answer is a 'Score Distribution' bar chart. The bottom section displays a document snippet titled 'Why parents should immunized or Vaccinated they're kids' with a source link to www.vaers.hhs.gov.



9. الميزة الإضافية: الاسترجاع المعزز بالتوليد (RAG)

- **الهدف:** تقديم استجابات دقيقة واضحة تجيب مباشرة على تساؤل المستخدم بدلاً من سرد المصادر والوثائق فقط.
- لإنتاج الإجابة (LLM) إلى نموذج لغوي كبير (Context) **نهج التنفيذ:** تمرير الوثائق المسترجعة الأفضل كـ
- HuggingFace أو OpenAI واجهات النماذج اللغوية المتاحة مثل / LangChain: **المكتبات المستخدمة**
- بدقة لضمان استنباط الإجابة من الوثائق المقدمة (Prompt Engineering) **قرارات التصميم المهمة:** ضبط التعليمات حصراً، لمنع ظاهرة التهلوس.

```

result = self.retriever.retrieve(query, top_k)

# 2. Fetch full text for context
context_texts = []
for d in result:
    text = self.document_store.get_document_text(d.doc_id)
    if text:
        context_texts.append(text[:500])

context = " ".join(context_texts)

# 3. Generate Answer
try:
    if self.llm_generator:
        prompt = f"Answer the question using ONLY the context below.\n\nContext:\n{context}\n\nQuestion:\n{query}\n\nAnswer:\n"
        res = self.llm_generator(prompt, max_new_tokens=50)
        answer = res[0]['generated_text'].strip()
    else:
        answer = f"[LLM Offline Mode] Retrieved Context Summary: {context[:200]}..."
except Exception:
    answer = f"[LLM Offline Mode] Retrieved Context Summary: {context[:200]}..."

return RagResult(
    answer=answer,
    context=context,
    documents=result
)

```

(RAG) ميزة الاسترجاع المعزز بالتوليد

(Question Answering) هذه الميزة تمثل الانتقال بالنظام من مجرد أداة بحث كلاسيكية إلى أداة لتقديم المعرفة وإجابة الأسئلة.

تقدم قائمة بالوثائق المقترحة، مما يثقل كاهل المستخدم بضرورة (Keyword Retrieval) المشكلة: محركات البحث التقليدية تصفح الوثائق وقراءتها لاستخراج الإجابة. وهذا غير كافٍ ومضيع للوقت، خصوصاً للاستعلامات المعقدة

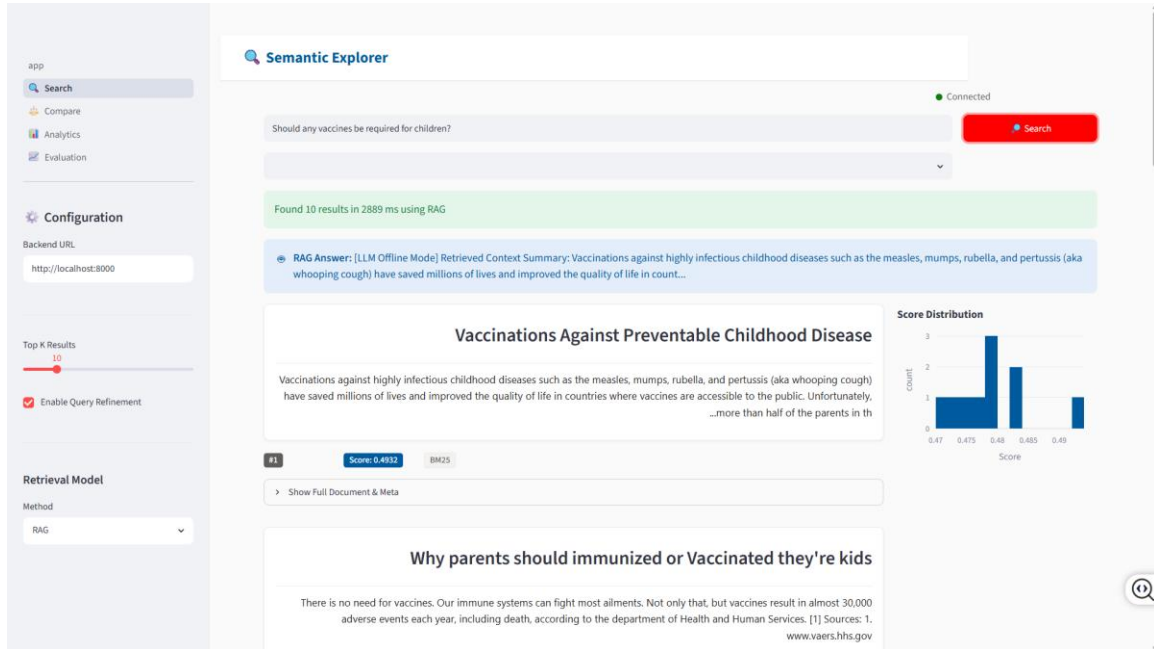
في فهم وقراءة النصوص، وتوجيه هذه (LLMs) لاستغلال قوة النماذج اللغوية الكبيرة (RAG) الحل المطبق: تطبيق نظام النماذج بناءً على الوثائق الدقيقة المسترجعة من نظامنا، مما يضمن دقة المعلومة المقدمة وموثوقيتها

(Pipeline) مسار العمل المعتمد

1. إدخال الاستعلام بصيغة سؤال طبيعي عبر الواجهة: **استعلام المستخدم (User Query)**
2. جلب الوثائق الأكثر صلة (Vector Store) الاستعلام من الفهارس والـ: **الاسترجاع (Retrieval)**
3. دمج نصوص الوثائق المسترجعة لتشكيل مرجع معرفي قوي: **بناء السياق (Context Construction)**
4. تغذية الاستعلام والـ السياق معاً إلى النموذج اللغوي لإنتاج الإجابة المخصصة: **التوليد (LLM Generation)**
5. عرض الإجابة البشرية بشكل فوري: **الاستجابة النهائية (Final Response)**

تفاصيل التنفيذ الفنية

- لفهرسة وتخزين المتجهات لضمان استرجاع أسرع وأدق للقطع النصية: **(Vector Store)** استخدام قاعدة المتجهات (Chunks).
- لضمان التركيز وتقليل (Top-K) يتم تحديده بعدد معين من الوثائق: **(Retrieved Context)** السياق المسترجع التشويش على النموذج اللغوي (وكذلك تقليل استهلاك التوكنز)
- تصميم واجهة شبيهة بالردشة التفاعلية لتقديم الإجابات بشكل بيهي: **(Chat-like Interface)** واجهة المحادثة واحترافي
- توفير نقطة نهاية واجهة برمجة تطبيقات: **(Independent Testing Method)** طريقة الاختبار (API) بمعزل عن باقي مكونات النظام أو واجهة المستخدم Prompt نتيج اختباره وقياس أداء الـ RAG خاصة للـ (Endpoint)



التقييم

يُعتبر التقييم المنهجي والمستقل أساس هذا المشروع لإثبات كفاءة النظام علمياً وعملياً

منهجية التقييم

(Ground Truth). تم اختبار كل نموذج بشكل منفصل لضمان الموثوقية والمقارنة العادلة، والاعتماد التام على التقييمات المعيارية

- المقدمة مع مجموعة البيانات بشكل قطعي (qrels) تم استخدام جميع استعلامات
- المحملة في النظام: [2210] (qrels) إجمالي عدد تقييمات
- إجمالي عدد الاستعلامات التي تم تقييمها فعلياً: [49]

المقاييس والنتائج الموثقة

1. مقياس (MAP - Mean Average Precision):

- التعريف: يعكس متوسط الدقة للاستعلامات، ويعطي وزناً أكبر لنجاح النظام في وضع الوثائق ذات الصلة في أعلى قائمة النتائج.
- النتائج:

النموذج	MAP Score
TF-IDF / VSM	0.0869
BM25	0.2005
Embeddings	0.1134
Hybrid Retrieval	0.1895

2. مقياس (nDCG - Normalized Discounted Cumulative Gain):

- التعريف: يُقيّم جودة الترتيب بناءً على تفاوت درجات الصلة للوثائق، مع مكافأة النظام عندما يضع النتائج الأكثر دقة في المراتب الأولى.
- النتائج:

النموذج	nDCG Score
TF-IDF / VSM	0.1344
BM25	0.32
Embeddings	0.16
Hybrid Retrieval	0.27

3. مقياس (Recall - الاستدعاء):

- التعريف: يوضح قدرة النظام على إيجاد واسترجاع كافة الوثائق الصحيحة والمتعلقة بالاستعلام من إجمالي الوثائق الموجودة في قاعدة البيانات.
- النتائج:

النموذج	Recall Score
TF-IDF / VSM	0.4
BM25	0.55
Embeddings	0.41
Hybrid Retrieval	0.55

4. (الدقة في أول 10 نتائج - Precision@10) مقياس:

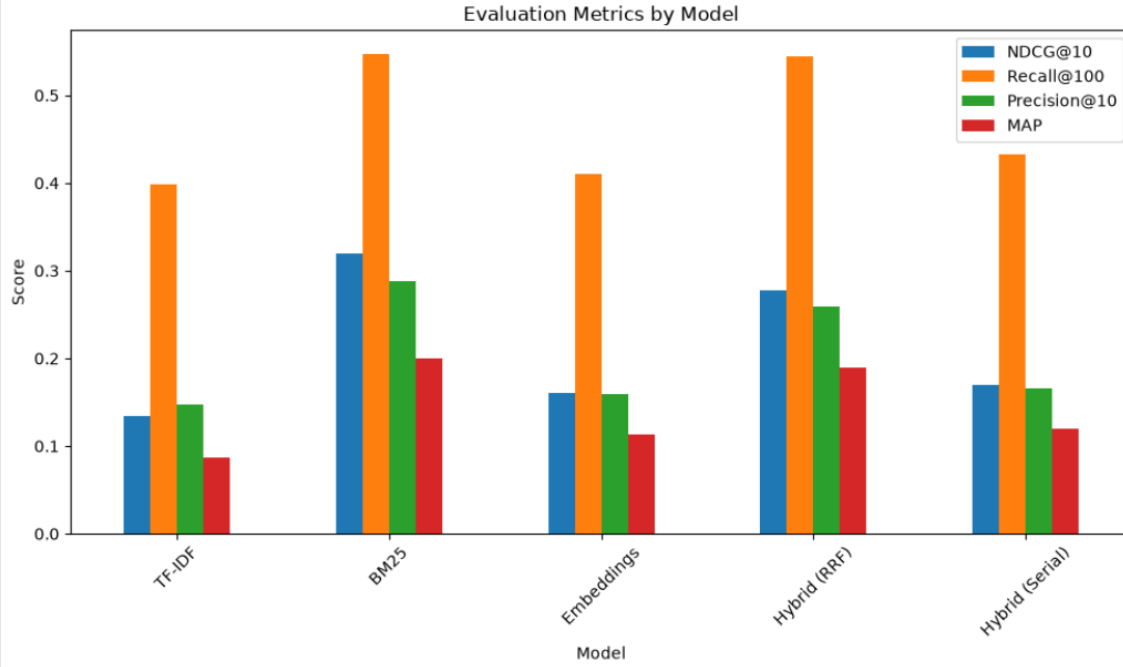
- **التعريف:** يحدد نسبة الوثائق الصحيحة وذات الصلة المباشرة التي تظهر للمستخدم ضمن أول 10 نتائج فقط
- **النتائج:**

النموذج	P@10 Score
TF-IDF / VSM	0.15
BM25	0.29
Embeddings	0.16
Hybrid Retrieval	0.26

Total qrels loaded: 2214
Total queries used: 49

Evaluation Metrics:

Model	NDCG@10	Recall@100	Precision@10	MAP
TF-IDF	0.1344	0.3987	0.1469	0.0869
BM25	0.3199	0.5469	0.2878	0.2005
Embeddings	0.1606	0.4110	0.1592	0.1134
Hybrid (RRF)	0.2782	0.5447	0.2592	0.1895
Hybrid (Serial)	0.1696	0.4330	0.1653	0.1204



القسم الثامن: المناقشة

نقاط القوة (Strengths):

- مما يضمن قابلية عالية للصيانة والتوسع، (SOA) بنية برمجية صلبة ومرنة بفضل
- الاسترجاع الهجين أثبت قدرته على سد ثغرات البحث اللفظي من خلال استيعاب سياق الاستعلام
- نقل النظام إلى مستوى التطبيقات الذكية العصرية بكفاءة عالية RAG تطبيق تقنية

القيود (Limitations):

- ومكالمات النماذج اللغوية (Embeddings Extraction) تكلفة حوسبة أعلى بسبب استخراج المتجهات
- RAG وسرعة استجابة - Context Window الحاجة لموازنة بين حجم السياق

الدروس المستفادة (Lessons Learned):

- أثبت عملياً قصور النماذج اللفظية البحتة في فهم النوايا الضمنية (qrels) الاعتماد على مقاييس أداء موضوعية عبر
- جودة الفهرسة والتنظيف المسبق للبيانات تؤثر بشكل هائل على استقرار نتائج البحث

التحسينات المستقبلية (Future Improvements):

- (Cross-Encoders Re-ranking) استخدام خوارزميات الذكاء الاصطناعي الأكثر تقدماً في عملية إعادة الترتيب Precision@10 لرفع مقياس
- استضافة النظام على بنية تحتية سحابية موزعة لرفع سرعة المعالجة المتزامنة وتقليل زمن الاستجابة

القسم العاشر: الخاتمة

(SOA) قدم هذا المشروع بنجاح محرك استرجاع معلومات متقدم، مبني على أسس هندسة البرمجيات السليمة عبر بنية الخدمات مقارنة بالأنظمة (Recall) و (MAP) وقد أظهرت نتائج التقييم المنهجية تفوقاً واضحاً لتقنيات الاسترجاع الهجين في مقياس الكلاسيكية البحتة

تأثيراً جذرياً في وظائف النظام؛ محولةً إياه من نظام تقليدي (RAG) علاوة على ذلك، أحدثت تقنية الاسترجاع المعزز بالتوليد يسرد قوائم من الروابط والمقاطع، إلى مساعد ذكي يقدم إجابات تحليلية واضحة تستند حصرياً إلى مصادر موثوقة ومفهرسة محلياً بناءً على هذه النتائج المعمارية والتقييمية، يمكن اعتبار النظام جاهزاً فنياً للاختبارات النهائية، ويمثل نواة قوية للتطبيق في النظم المؤسسية التي تتطلب فهماً عميقاً للبيانات غير المهيكلة
